

HealkrISTin: compreender a base e explicar as tasks

Guia para cerca de duas horas de preparação. Usa o código novo desta pasta, não os módulos `clusters.c` e `tarefas.c` dos ZIPs antigos. A base compila em C99, lê e valida os dados e termina antes de resolver as tasks. A referência é o enunciado fornecido, versão 1.0, sobretudo as secções 1.2, 2, 3, 4 e 6.2. As escolhas que o enunciado não define são identificadas.

1. Plano de duas horas e primeira execução

Minutos	O que fazer	O que deves conseguir explicar
0-10	Ler secções 2 e 3; correr <code>make demo</code>	Cidades, ligações e os três ficheiros
10-30	Abrir <code>main.c</code> e acompanhar a secção 4	A ordem de execução e os erros
30-55	Abrir <code>grafo.h</code> e <code>grafo.c</code> ; secções 5 e 6	Structs, ponteiros, listas e memória
55-75	Ler <code>leitura.c</code> , <code>perguntas.c</code> ; secções 7 e 8	Validação e pedidos mal definidos
75-100	Estudar secções 10 a 13	DFS, Task1, Task2 e Task3 versus Task4
100-110	Ler secções 9 e 14	Makefile e complexidades
110-120	Responder à secção 16 sem consultar	Explicação curta, com exemplos

Começa por extrair o ZIP e abrir um terminal na pasta do Makefile:

```
make
make demo
```

Vais ver 9 cidades e 7 ligações, as posições, os vizinhos e cada pedido. Task3 21 e Task4 0 são mal definidos: neste exemplo os IDs vão de 1 a 9. Verás a expressão “formato e ID válidos” para pedidos bem escritos; isso ainda não garante que a task seja possível no grafo.

Para o programa normal:

```
./healkristin exemplos/enunciado.quests exemplos/enunciado.map exemplos/enunciado.position
```

Não aparecer nada é esperado. A base normal é silenciosa e não cria `.results`, porque não existem respostas calculadas. `make demo` é uma ferramenta de estudo que imprime os dados num executável separado. É necessário GCC e Make; o programa não depende de Python.

2. O problema em palavras simples

Imagina cidades ligadas por cabos que sobreviveram à tempestade. Algumas ficaram em grupos separados. Precisamos de saber os grupos e quais as cidades que ficam geometricamente mais próximas de outros grupos.

Uma cidade corresponde a um vértice. Uma ligação física corresponde a uma aresta. O conjunto é um grafo não orientado: uma ligação 1-2 pode ser percorrida de 1 para 2 e de 2 para 1.

Um cluster é uma componente conexa: um conjunto máximo de cidades que conseguem chegar umas às outras seguindo ligações, diretas ou indiretas. Se existem 1-2 e 2-3, as três cidades pertencem ao mesmo cluster, mesmo sem uma ligação direta 1-3. Uma cidade isolada forma um cluster de tamanho um. Não deixa de existir por não aparecer numa aresta.

Os clusters vêm das ligações do `.map`. As distâncias vêm das coordenadas do `.position`. Duas cidades próximas podem estar em clusters diferentes; duas cidades afastadas podem pertencer ao mesmo cluster. A primeira fase não recebe uma lista de cidades com produção de energia: analisa a conectividade física do mapa recebido.

Task	Pergunta
Task1	Quantos clusters existem?
Task2	Quais as cidades de cada cluster?
Task3 r	Qual a cidade de outro cluster mais próxima da cidade r?
Task4 r	Qual a cidade externa mais próxima de qualquer cidade do cluster de r?

Uma frase possível na oral: “Represento a rede por um grafo não orientado com listas de adjacência. Leio as coordenadas para um vetor. A base valida os dados; depois seria necessário calcular as componentes conexas e usar essas componentes nas quatro tasks.”

3. Ler os três ficheiros

A invocação tem esta ordem obrigatória:

```
./healkristin pedidos.quests rede.map locais.position
```

Os nomes podem ser diferentes. As extensões e a ordem têm de ser estas. `.map.txt` não termina em `.map`; `.MAP` não é `.map`.

O `.map` começa com C e L: número de cidades e número de ligações. As cidades existem com IDs 1 a C, incluindo as que não têm ligações. Seguem-se exatamente L pares de cidades.

```
4 1
1 2
```

Há quatro cidades e uma ligação 1-2. As cidades 3 e 4 estão isoladas. O mapa tem três clusters: {1,2}, {3}, {4}.

O `.position` começa com Xmax e Ymax, os limites do plano. Não é um cabeçalho C L e não há uma linha com o número de cidades. Depois, cada linha contém ID, x, y:

```
110 10
1 1 1
2 101 1
3 3 1
4 102 1
```

Todas as coordenadas são inteiras e naturais: x e y começam em 1. É necessário $1 \leq x \leq X_{\max}$, $1 \leq y \leq Y_{\max}$ e um registo por cidade. Esta base aceita IDs em qualquer ordem e rejeita IDs repetidos.

O `.quests` tem uma instrução por linha:

```
Task1
Task2
Task3 1
Task4 1
```

Task1 e Task2 não recebem argumentos. Task3 e Task4 recebem uma cidade. Linhas vazias são ignoradas nesta implementação.

4. Seguir o main do início ao fim

main é o coordenador. Pede trabalho aos módulos e limpa os recursos. Não deve conter toda a representação do grafo nem todos os algoritmos.

1. Inicializa ponteiros a NULL e o resultado a EXIT_FAILURE.
2. Verifica argc == 4 e as três extensões.
3. Abre os três ficheiros com fopen(..., "r").
4. Chama leitura_mapa: recebe um Grafo ou NULL.
5. Chama leitura_posicoes: valida e guarda todas as coordenadas.
6. Prepara a string do futuro nome .results, sem abrir esse ficheiro.
7. Lê e valida cada pergunta, uma de cada vez.
8. Quando chega a EOF sem erro, define EXIT_SUCCESS.
9. Fecha ficheiros e liberta memória em fim:.

Porque argc é 4? argv[0] contém o nome do programa. Os três argumentos estão em argv[1], argv[2] e argv[3]. argc conta o programa e os argumentos.

O operador || faz curto-circuito. Se argc != 4 for verdadeiro, não avalia as verificações seguintes nem tenta aceder a argumentos que podem não existir. O operador && também faz curto-circuito, parando quando a primeira condição falsa torna o resultado falso.

fopen devolve NULL se não conseguir abrir. Não podemos ler de um FILE * NULL. Os erros levam a goto fim, um ponto comum de limpeza. Aqui, goto evita repetir todos os fclose e free em cada falha. Não substitui a estrutura dos algoritmos.

EXIT_FAILURE e EXIT_SUCCESS são códigos de saída, não mensagens. O terminal só recebe texto se o programa chamar algo como printf. No modo normal não existem essas chamadas ativas.

O comentário "PARAR AQUI A LOGICA DE RESOLUCAO" indica onde se poderá acrescentar o pré-processamento dos clusters. Dentro do ciclo das perguntas poderá depois entrar a escolha da task e a escrita da resposta. Essa parte está deliberadamente por implementar.

5. .h, .c e tipo abstrato de dados

Módulo	O que guarda ou faz
grafo	Ligações e coordenadas; criar, consultar e destruir
leitura	Transformar .map e .position num grafo válido
perguntas	Transformar cada linha .quests numa Pergunta
util	Leitura de texto, conversão de inteiros, nomes e alocação
main	Coordenar tudo

Um .h anuncia os tipos e as funções disponíveis. Um .c explica como as funções funcionam. O compilador precisa das declarações para verificar uma chamada. O linker junta depois as implementações compiladas.

#include "grafo.h" inclui o cabeçalho. Não inclui automaticamente grafo.c. Não se deve fazer #include "grafo.c" nesta organização.

```
#ifndef GRAFO_H
#define GRAFO_H
/* declaracoes */
#endif
```

Estas guardas impedem o processamento repetido do mesmo cabeçalho numa unidade de compilação. Uma unidade de compilação é, simplificando, um .c com os cabeçalhos que inclui.

```
typedef struct Grafo Grafo;
```

Esta linha diz que existe um tipo Grafo, mas não mostra os seus campos. O struct Grafo completo está em grafo.c. Os outros módulos trabalham com Grafo * e funções públicas, como grafo_num_cidades. Não conseguem escrever diretamente g->numero_cidades. Isto é um tipo abstrato com representação escondida.

Ligacao e Posicao têm campos públicos no .h: os percursos precisam de ler os vizinhos e as coordenadas. A estrutura principal Grafo é opaca. Uma função static, como identificar em perguntas.c, só é visível dentro desse .c. Serve como auxiliar privado.

6. Ponteiros, structs e listas de adjacência

Uma struct agrupa campos relacionados. Posicao agrupa x e y. Ligacao agrupa um vizinho e o ponteiro para o próximo nó.

```
typedef struct Ligacao {
    int destino;
    struct Ligacao *seguinte;
} Ligacao;
```

int *p é um ponteiro para um int. *p acede ao valor apontado. &n obtém o endereço de n. g->campo acede a um campo através de um ponteiro; equivale a (*g).campo. p.x usa uma struct por valor. NULL representa a ausência de um objeto apontado.

No grafo, Ligacao **adjacencias aponta para um vetor de ponteiros. Cada adjacencias[cidade] é a cabeça da lista daquela cidade. O campo seguinte aponta para o nó seguinte; NULL marca o fim.

Para ligações 1-2, 2-3 e 4-5, a representação lógica é:

Cidade	Vizinhos
1	2
2	1 e 3
3	2
4	5
5	4

Cada ligação é guardada duas vezes. São duas entradas de uma única aresta não orientada. Existem C cabeças e 2L nós: memória $O(C + L)$. Uma matriz de adjacência usa $C \times C$ entradas: memória $O(C^2)$.

Como se insere 1-2? Alocam-se dois nós. No nó da lista de 1, guarda-se 2. No nó da lista de 2, guarda-se 1. Só se alteram as listas depois de ambas as alocações terem sucesso.

```
ida->destino = b;
ida->seguinte = g->adjacencias[a];
g->adjacencias[a] = ida;
```

A segunda linha liga o nó novo à antiga cabeça. A terceira torna o novo nó a cabeça. Os vizinhos anteriores não se perdem. Não procuramos o fim da lista: a inserção demora $O(1)$. A ordem de leitura fica invertida na lista, mas a conectividade é a mesma.

Os vetores têm $C+1$ posições. A posição 0 fica sem uso para podermos usar diretamente IDs de 1 a C. Consultar uma posição custa $O(1)$. Percorrer os vizinhos de uma cidade custa $O(\text{grau da cidade})$.

`const Ligacao *`no permite consultar o nó através daquele ponteiro. É um ponteiro emprestado: quem o recebe não deve libertá-lo. `const Grafo *` indica uma operação de consulta; o contrato é não alterar o grafo, mesmo que `const` em C não imponha imutabilidade profunda.

7. Ler e validar mapa e coordenadas

Em `leitura_mapa`, o cabeçalho exige exatamente dois inteiros: $C > 0$, $L \geq 0$. O ciclo lê exatamente L pares, validados em `grafo_adicionar_ligacao`. Não admite uma cidade 0 ou $C+1$. Também não admite arestas adicionais depois das L declaradas nem dados em falta.

Uma rede com $L = 0$ é válida. Os C vértices existem, todos com listas vazias. Arestas repetidas e auto-ligações são aceites nesta base; uma futura pesquisa com marcação não repetirá cidades por isso.

Em `leitura_posicoes`, o cabeçalho define limites positivos. Cada linha tem exatamente três inteiros. Valida-se ID antes de o usar como índice. Depois validam-se as coordenadas e a ausência de repetições.

```
vistas = util_alocar_vetor((size_t)C + 1, sizeof(*vistas), 1);
```

O último argumento 1 escolhe `calloc`: todos os bytes começam a zero. `vistas[3] == 0` significa que a cidade 3 ainda não apareceu. Quando aparece, marca-se 1. Uma segunda ocorrência é rejeitada.

Ao terminar, `lidas == C` mais IDs distintos em 1 a C garante que todas as cidades apareceram. Não basta contar linhas sem detetar repetições: poderias ter C linhas e faltar uma cidade.

Coordenadas inválidas interrompem a execução. Exemplos: zero, negativas, maiores que os limites, decimais, cidade inexistente ou cidade em falta. Não se pode usar simplesmente (0,0) para uma cidade ausente.

`util_inteiros` usa `strtoul`. Verifica onde termina a conversão, se houve `ERANGE`, se o valor cabe em `int` e se há texto extra. `atoi("abc")` não permite distinguir esse erro de um zero legítimo. Aqui, 3.5, 2abc e campos a mais são rejeitados.

`util_ler_linha` cresce o buffer com `realloc` e termina-o com `\0`. Lê a última linha mesmo sem `newline` e aceita CRLF: o CR é tratado como espaço. O retorno 1 significa linha, 0 EOF e -1 erro; não confundir EOF normal com uma falha de leitura.

8. Reconhecer perguntas sem as executar

Pergunta guarda o tipo, a cidade, a validade preliminar e o texto. `TipoTask` é uma enumeração, dando nomes a valores inteiros: `TASK_DESCONHECIDA` vale 0; `TASK1` a `TASK4` valem 1 a 4.

`identificar` exige o nome exato, seguido de fim de linha ou espaço. Assim, `Task10` e `Task1abc` não são confundidas com `Task1`.

Pedido	Validação nesta base
Task1	Válido, sem argumentos
Task2 8	Mal definido: argumento a mais
Task3 5	Válido se 5 estiver em 1 a C
Task3	Mal definido: falta a cidade
Task4 abc	Mal definido: não é inteiro
Task4 0	Mal definido: cidade inexistente
Task5	Desconhecido nesta base da primeira fase

Uma pergunta mal definida não é um erro fatal de leitura. `perguntas_ler` devolve 1 e `marca_valida = 0`. O `main` continua a ler. Quando as `tasks` existirem, essa linha deverá receber a resposta -2. A base apenas mostra a classificação no modo demo.

Se `Task3` 1 tem formato e ID válidos mas o grafo tem só um cluster, o pedido é impossível. Essa condição só será verificada depois de calcular clusters; não faz parte de `valida` nesta versão.

`pergunta.texto` aponta para dentro do buffer `linha`. Não é uma cópia independente e deixa de poder ser usado depois da leitura seguinte. Por isso processamos um pedido de cada vez e não acumulamos ponteiros para perguntas antigas. `free` deve receber `linha`, não `pergunta.texto`.

O futuro ficheiro `.results` herdará o caminho e nome do `.quests`: `pasta/pedido.quests` dá `pasta/pedido.results`. As respostas deverão seguir a ordem dos pedidos, com uma linha em branco entre blocos.

9. Memória e Makefile

`malloc` reserva memória sem a inicializar. `calloc` reserva e põe os bytes a zero. `realloc` pode mudar o endereço de uma alocação. `free` devolve memória ao sistema. `sizeof(*p)` mede um elemento do tipo de `p`. `size_t` é o tipo próprio para tamanhos de objetos e de buffers.

Usa-se um ponteiro temporário para `realloc`. Se falhar, a alocação antiga continua válida. Escrever diretamente `linha = realloc(linha, tamanho)` poderia perder o endereço antigo se o retorno fosse `NULL`.

Recurso	Quem o liberta
Grafo, cabeças, posições e nós	<code>grafo_destruir</code> , chamado pelo <code>main</code>
Linha temporária de mapa/posições	A própria função de leitura
Vetor vistas	<code>leitura_posicoes</code>
Buffer das perguntas	<code>main</code>
String do nome de resultados	<code>main</code>
Ficheiros abertos	<code>main</code> , com <code>fclose</code>

Ao destruir uma lista, guarda-se `atual->seguinte` antes de `free(atual)`. Depois de `free`, não se pode ler o nó. `free(NULL)` é permitido; `fclose(NULL)` não é, pelo que se verifica o ponteiro.

O `Makefile` permite compilar só o que mudou. Cada `.c` gera um `.o`; o linker junta os objetos no executável.

```
%.o: %.c
    $(CC) $(CPPFLAGS) $(CFLAGS) -MMD -MP -c $< -o $@
```

`$<` é a primeira dependência e `$@` o alvo. `-c` não faz a ligação. `-MMD -MP` cria ficheiros `.d` com dependências dos cabeçalhos. Alterar um `.h` também provoca a recompilação dos `.c` que o incluem.

`-std=c99` seleciona C99; `-Wall -Wextra -Wpedantic` ativam avisos; `-O0 -g` facilita estudar com um depurador. As linhas de comandos começam com TAB. Espaços podem causar o erro “missing separator”.

`make demo` define `DEMONSTRAR` ao compilar um executável separado. Assim, os `#ifdef DEMONSTRAR` entram só nesse programa. `make clean` remove executáveis, objetos e dependências, conservando o código.

10. Task1: contar componentes com DFS

Esta secção explica o algoritmo futuro. Não existe implementação de DFS, BFS ou Task1 nos ficheiros C entregues.

Usa um vetor `grupo[cidade]`, inicialmente -1, e um contador `K`. Cada cidade recebe o identificador da componente a que pertence.

```
K = 0; todas as cidades ainda sem grupo
para cada cidade r, em ordem crescente:
    se r ainda nao tem grupo:
        marcar r com grupo K e inserir r na pilha
        enquanto a pilha nao estiver vazia:
            retirar a ultima cidade inserida
            para cada vizinha dessa cidade:
                se ainda nao tem grupo:
                    marcar com K e inserir na pilha
        K = K + 1
```

DFS significa pesquisa em profundidade. A pilha é LIFO: a última cidade a entrar é a primeira a sair. Pode ser um vetor de `C` inteiros com um índice topo, evitando muitas chamadas recursivas numa cadeia longa.

Marca-se quando a cidade entra na pilha. Assim, dois vizinhos não a inserem duas vezes. Uma pesquisa visita exatamente uma componente; uma cidade ainda sem marca após a pesquisa inicia outra componente.

No mapa com 1-2, 2-3, 4-5 e cidade 6 isolada: a pesquisa a partir de 1 marca {1,2,3}; a de 4 marca {4,5}; a de 6 marca {6}. $K = 3$. A saída futura é Task1 3.

Cada cidade é descoberta uma vez e cada nó de adjacência é percorrido uma vez. São `C` cidades e `2L` nós: tempo $O(C + L)$, memória auxiliar $O(C)$. BFS com fila também resolve em $O(C + L)$. Union-Find é outra alternativa para agrupar durante a leitura, mas não é necessário para esta base.

As Task1-Task4 não modificam o mapa. Podemos calcular os grupos uma única vez antes das perguntas. Depois, responder Task1 usando `K` é $O(1)$. Sem esse pré-processamento, calcular a resposta custa $O(C + L)$.

11. Task2: listar os clusters pela ordem pedida

Task2 usa as mesmas componentes da Task1, mas também imprime os membros. No exemplo de seis cidades, a saída futura seria:

```
Task2 3
Cluster: 1 2 3
Cluster: 4 5
Cluster: 6
```

Existem duas regras: cidades em ordem crescente dentro de cada cluster; clusters em ordem crescente da menor cidade de cada um. A ordem de descoberta da DFS não garante a primeira regra.

O ciclo exterior da DFS começa sempre pela menor cidade ainda sem marca. Por isso, os IDs de componentes descobertos assim já respeitam a ordem dos mínimos. Falta organizar os membros.

Uma opção simples: percorrer cidades de 1 a `C` e acrescentar cada cidade à lista de membros do seu grupo. Usando uma cauda por lista, cada acrescento custa $O(1)$, e as listas ficam ordenadas sem ordenar depois. Outra opção é um vetor de membros dividido em intervalos por cluster, com tamanhos e offsets, como na solução antiga.

Depois da organização, imprimir tudo visita `C` IDs: tempo $O(C)$, além dos caracteres escritos, e memória de membros $O(C + K)$. Organizar os grupos também pode custar $O(C + K)$.

Uma solução com “para cada cluster, varrer todas as `C` cidades” é correta na ordem, mas demora $O(KC)$, chegando a $O(C^2)$. Não se deve dizer que esse ciclo duplo custa $O(C)$. A estrutura escolhida altera a complexidade.

Um cluster com uma única cidade é listado normalmente. Task1 e Task2 não precisam de coordenadas para calcular os grupos, embora a validação global do programa deva rejeitar um `.position` inválido.

12. Task3: proximidade à cidade de referência

Para uma referência r , consideram-se apenas cidades fora do cluster de r . Escolhe-se a que está mais próxima de r em linha reta no plano. Não é o caminho mais curto através de arestas do `.map`.

A distância euclidiana é a raiz de:

$$q(a,b) = (x_a - x_b)^2 + (y_a - y_b)^2$$

$$d(a,b) = \sqrt{q(a,b)}$$

Para escolher a menor distância basta comparar q : a raiz quadrada preserva a ordem para números não negativos. Task3 e Task4 pedem uma cidade, não pedem que se escreva o valor da distância.

```
se r nao existe ou K == 1: resposta -2
melhor cidade = -2; melhor distancia = infinito
para cada cidade b fora do grupo de r:
    calcular q(r,b)
    se for melhor: guardar b e a distancia
devolver a melhor cidade
```

Há no máximo C candidatas: $O(C)$ por consulta nesta abordagem direta. Não precisamos de uma matriz com todas as distâncias, de memória $O(C^2)$. Calcular cada distância custa $O(1)$.

As coordenadas cabem em `int`, mas o quadrado de uma diferença pode não caber. Por exemplo, 100000^2 já excede um inteiro com sinal de 32 bits. Numa implementação futura, converte-se para 64 bits antes da subtração e da multiplicação; converter depois não corrige um overflow já ocorrido. Na faixa positiva de `int` de 32 bits, uma soma em `uint64_t` é suficiente.

Se houver empate, o enunciado fornecido não define o desempate. Uma escolha documentada é o menor ID externo. Percorrer os IDs em ordem crescente e só atualizar quando a distância for estritamente menor produz essa regra para Task3.

Se não existe cidade externa, não é uma distância zero: é um pedido impossível, com resposta -2. Cidades distintas com as mesmas coordenadas podem ter distância zero válida, se estiverem em clusters diferentes.

13. Task4: proximidade ao cluster inteiro

Task4 r usa r apenas para identificar o seu cluster S . Depois compara todas as cidades de S com todas as cidades fora de S . A resposta é a cidade externa do par com menor distância.

```
se r nao existe ou K == 1: resposta -2
melhor cidade externa = -2; melhor distancia = infinito
para cada cidade externa b, por ordem crescente:
    para cada cidade a do cluster S:
        calcular q(a,b)
        se for estritamente menor: guardar b e a distancia
devolver a melhor cidade externa
```

Este pseudocódigo adota o menor ID externo em empates. A ordem crescente do ciclo exterior e a atualização estrita garantem esse desempate. A resposta não é a cidade interna a , nem o ID do cluster de b .

Usa o exemplo incluído em `exemplos/diferenca.*`:

Cidade	Posição	Cluster
1	(1,1)	A
2	(101,1)	A
3	(3,1)	B
4	(102,1)	C

Só há ligação 1-2. Para Task3 1, a cidade 3 fica a 2 de 1 e a cidade 4 fica a 101 de 1. Resposta: Task3 1 3.

Para Task4 1, também olhamos para a cidade 2, que é do cluster A. A cidade 4 fica a 1 da cidade 2. A distância mínima de A a 3 é 2; a distância mínima de A a 4 é 1. Resposta: Task4 1 4.

Task4 2 também dá 4, porque o cluster é o mesmo. Task3 1 e Task3 2 podem dar cidades diferentes, pois usam referências diferentes.

Se S tem s cidades, há $s(C-s)$ pares. Varrer os C IDs para identificar externas acrescenta $O(C)$: custo $O(C + s(C-s))$, pior caso $O(C^2)$. Uma primeira consulta de Task4 não é, em geral, $O(C)$.

No exemplo de nove cidades do enunciado, os grupos são $\{1,2,3,6,7\}$ e $\{4,5,8,9\}$. Há $K=2$. Task3 5 escolhe 6. Task4 7 pode escolher 4 com o desempate por menor ID externo: 3-4 e 2-9 têm distância 1. Não confundir esse exemplo de entrada com a figura de 20 cidades no PDF.

14. Complexidades, erros e decisões a justificar

Usa C para cidades, L para ligações, K para clusters, s para tamanho do cluster consultado, B para maior linha e T para caracteres de entrada.

Operação	Tempo
Construir/ler o mapa	$O(C + L + \text{texto do mapa})$
Ler posições	$O(C + \text{texto das posições})$
Ler todas as perguntas da base	$O(\text{texto das perguntas})$
Base completa	$O(C + L + T)$
DFS futura, uma vez	$O(C + L)$
Task1 depois da DFS	$O(1)$
Task2 com membros já organizados	$O(C)$, além da escrita
Task3 por consulta direta	$O(C)$
Task4 por consulta direta	$O(C + s(C-s))$, pior caso $O(C^2)$

Memória da base: $O(C + L + B)$. Guardamos C cabeças, $2L$ nós, C posições e um buffer reutilizado. Não guardamos todas as perguntas. Vetor de grupos e pilha acrescentariam $O(C)$, mantendo a mesma ordem.

Uma futura cache pode guardar Task3 por cidade e Task4 por cluster. Uma consulta repetida passaria a $O(1)$ após o primeiro cálculo, além de ler/escrever texto. Isso funciona enquanto a rede e as posições não mudarem. As caches não estão implementadas nesta base.

O enunciado fornecido não dá limites numéricos gerais para C e L. Não se pode garantir que a pesquisa quadrática passa qualquer limite de tempo. A solução direta é simples de justificar; técnicas espaciais podem ser discutidas como otimizações futuras.

Situação	Comportamento da solução futura
Número/extensões de argumentos errados; ficheiro ausente	Terminar sem saída
Rede ou coordenadas inconsistentes	Terminar sem saída
Cidade inexistente num pedido	Repetir a instrução e acrescentar -2
Task3/Task4 com só um cluster	Repetir a instrução e acrescentar -2
Pergunta bem definida e possível	Escrever a resposta calculada

A base já faz os dois primeiros tratamentos e reconhece perguntas mal definidas. Não escreve os -2 nem as respostas normais. Não uses -2 como resposta temporária de todas as tasks: isso afirmaria que pedidos válidos são impossíveis.

15. Se perguntarem pelas Task5 e Task6

São da fase final; concentra a preparação nas quatro primeiras. Esta base também não implementa estes algoritmos.

Task5 procura o comprimento mínimo total de cabo para unir os clusters e os pares de cidades a ligar. Pode pensar-se num grafo em que cada vértice é um cluster. O peso entre dois clusters é a menor distância entre uma cidade de cada um, guardando também o par de cidades. Uma árvore geradora mínima nesse grafo, por Prim ou Kruskal, determina as ligações mínimas de restauro. Para K clusters, são K-1 novas ligações quando $K > 1$. Não basta escolher ligações locais sem evitar ciclos.

Os comprimentos somados têm de ser distâncias reais. Comparar quadrados serve para escolher pares e ordenar arestas, mas somar quadrados não devolve o comprimento total de cabo.

Task6 acrescenta a rota da equipa: partir da cidade indicada, executar cada ligação de restauro como um passo entre os seus extremos e voltar à cidade inicial. Há deslocações entre restauros e podem repetir-se cidades. Só visitar as cidades não garante executar as ligações exigidas. A ordem das arestas de uma MST não resolve automaticamente esta rota.

O enunciado apresenta Task6 com a distância adicional de deslocações, excluindo o comprimento de cabo instalado. No exemplo, 90 de restauro mais 25 de deslocação dão 115 de percurso total; a linha Task6 mostra 25. As linhas da rota seguem a ordem de passagem, enquanto Task5 ordena as ligações pelos IDs. As distâncias são escritas com duas casas decimais. Esta é uma descrição conceptual, não uma solução completa da Task6.

16. Treinar a oral sem decorar o texto

Responde primeiro sem olhar; usa depois as respostas para corrigir.

- O que está implementado?** Invocação, leitura e validação dos três ficheiros, grafo e posições, reconhecimento de perguntas e limpeza. Os algoritmos e as respostas das tasks estão por fazer.
- Porque é não orientado?** Porque a ligação física funciona nos dois sentidos; guardo dois nós por aresta.
- O que significa C+1?** Quero usar diretamente os IDs 1 a C; reservo a posição 0 sem a usar.

4. **Porque listas?** Ocupam $O(C+L)$ e permitem percorrer os vizinhos existentes. Uma matriz ocupa $O(C^2)$, mas consulta uma aresta em $O(1)$.
5. **O que faz cada nó?** Guarda o ID de um vizinho e o ponteiro seguinte. A cabeça de cada lista identifica a cidade de origem.
6. **Porque validar antes de indexar?** Uma cidade 0, negativa ou maior que C poderia causar um acesso fora do vetor.
7. **Para que serve vistas?** Detetar IDs repetidos e, com a contagem, garantir que existem coordenadas para todas as cidades.
8. **Porque não usar atoi?** Não fornece a mesma deteção de formato e overflow; strtol permite validar a conversão.
9. **Porque char **linha?** A função pode realocar e atualizar o endereço do buffer do chamador.
10. **Porque goto fim?** Centraliza a limpeza dos recursos nos vários caminhos de erro; a lógica das tarefas continuará em funções.
11. **Como contar clusters?** Iniciar uma DFS/BFS em cada cidade ainda não descoberta e contar quantas pesquisas são iniciadas.
12. **Uma cidade isolada conta?** Sim, como uma componente de um vértice.
13. **Porque marcar antes de inserir na pilha?** Para evitar inserções repetidas da mesma cidade por vizinhos diferentes.
14. **Task2 sai ordenada pela DFS?** Não necessariamente. Organizo os membros por IDs crescentes; os mínimos determinam a ordem dos grupos.
15. **Qual a diferença Task3/Task4?** Task3 fixa a cidade de referência; Task4 considera todo o seu cluster e devolve uma cidade externa.
16. **É preciso Dijkstra?** Não para Task1-Task4: usamos conectividade e distâncias geométricas, não caminhos mínimos com pesos.
17. **Porque evitar sqrt?** Só comparamos distâncias e a raiz preserva a ordem. Os quadrados têm de ser calculados num tipo suficiente.
18. **Quando usar -2?** Pergunta mal definida ou impossível. Dados inválidos de entrada global mandam terminar, sem resultados.
19. **Como compila?** Cada .c gera .o; o linker junta-os. Os .h fornecem declarações e tipos, não são executáveis independentes.
20. **Como sei o que a base leu?** Corro make demo e comparo cidades, posições, vizinhos e classificação dos pedidos com os ficheiros.

Explicação de cerca de um minuto:

“O programa recebe três ficheiros numa ordem fixa. O main verifica os argumentos, abre os ficheiros e pede aos módulos de leitura que construam um grafo válido. Uso listas de adjacência, guardando dois nós por ligação, porque a rede é não orientada. As coordenadas ficam num vetor por cidade. Depois leio as perguntas uma a uma e valido o seu formato e os IDs. Ainda não tenho as tasks implementadas. Para as acrescentar, calcularia as componentes conexas uma vez com DFS. Task1 usa a quantidade de grupos; Task2 lista os membros ordenados; Task3 procura a cidade externa mais próxima da referência; Task4 procura a externa mais próxima de qualquer cidade do grupo. Por fim, escreveria as respostas no results e libertaria os recursos.”

Exercícios rápidos: prevê os vizinhos de 2 após inserir 1-2 e 2-3; explica o que acontece se faltar a posição de 4; calcula Task3 1 e Task4 1 no exemplo de quatro cidades; explica onde libertas cada nó. Se consegues responder com o código aberto, já estás a justificar a implementação, em vez de apenas repetir nomes de funções.